Java Tabanlı Nesneye Yönelik Yaklaşımla K-En Yakın Komşu ve Karar Ağacı Algoritmalarının Karşılaştırmalı Analizi

Hasan Hüseyin Yüksel – HANI ASHRAFI
Bilgisayar Mühendisliği Bölümü
Kocaeli Üniversitesi
Kocaeli, Türkiye
yukselhuseyin58@gmail.com-haofficial122@gmail.com

Bu projemizde bizden istenen amaçlar doğrultusunda sınıflandırma kapsülleme gibi değerlerinide göz önünde bulundurarak Kocaeli satış excelinden çektiğimiz yaş şehir gibi verileri işleyerek ürün kategorisini temsil eden bir makine oluşturmaktı. Bunu yaparken veri çekmede olsun ve gui değişikliklerde olsa sonunda istenen sonuca ulaşılıp tam teşekkürlü sonuç yazısı, grafik tablosu ve ayrı modelleri bulunan bir gui ulaşarak proje amacına yüzde 90'a yakın doğruluk oranıyla ulaşılmıştır, Bu yaşanan zorluklardan dersimiz veri çekmenin ve gui özeliklerinin ne kadar önemli olduğudur ve ileri projelerde bunları kullanıcağımızdır.

I. GİRİŞ

Makine öğrenmesi simdiki zamanda herhangi bir grup veriyi analiz edip ogrenip sonraki verileri tahmin etmeye yariyor. Bu projenin amacı,gerçek bir veri grubunu analiz edip ,analiz eddikten sonra ürün kategorilerini tahmin eden bir sistem geliştiriyor. bu projede ai ve nyp kisminda bir hazir kutuphane kullanmadik , K En Yakın Komşu (KNN) ve karar Ağacı (Decision Tree) algoritmaları Javada kodlamaya calistik. Bu projede nyp ye cok ic ice gecen bir projedir , biz nyp kurallarina bagli kalmaya calistik; algoritmaların doğruluk oranları , bellek tüketimlerini ve çalışma sürelerini her iki algritmada karşılaştırmalı olarak analiz ettik ve gösterdik

II. Veri Seti ve Ön İşleme

Projede kullanılan veri seti, 2017 yılıinda kocaeli ili icinde satis verilerini (MarketSalesKocaeli.xlsx)gosteriyor. Projede bizden istenilen seyler boyledir:Veri Temizleme : Veri setindeki 5093 satır okunurken boş (null) değerler ve hedef kategorisi "UNKNOWN" olan satırlar ayıkladik ve sildik.Veri Bölme : Veri seti rastgele secilmesi; %80 Eğitim ve %20 Test olarak ikiye ayrılmıştır. Veri sızıntısını önlemek için bu işlem eğitimden önce yapiliyor ki o verinin sayiisi belli olsun.Sayısal Dönüştürme : Kategorik bir veri olan Cinsiyet, mesafe hesaplamalarında kullanılabilmesi için erkek ve kadini

normalize etmemzi gerekiyordu ;Erkek=1.0, Kadın=0.0. Ayrıca kategorik "Marka" bilgiside kullanin denildigi icin algoritmalarda yer almaktadir.Normalizasyon ve Standardizasyon Analizi: Farklı ölçeklerdeki verilerin 1 ve 0 arasinda degeralmasini saglar ama standardizasyon bunu eksi yada 1 den fazla olmasini saglar ama projede bizden istenilen normalizasyon oldugu icin biz projede standardizasyon kullanmadik. KNN algoritmasını hesaplamak icin(0 ve 1 arasi olacagi icin) Min-Max Normalizasyon formülü kullandik:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

kullanılarak veriler 0 ve 1 aralığına çekilmiştir.eger standarizasyona izin verilseydi onun icin kullanacagimiz formul boyledir:

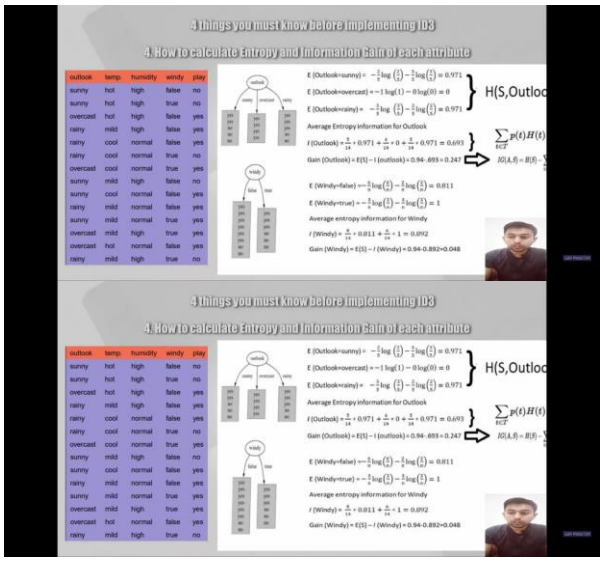
$$X_{stand} = \frac{X - \mu}{\sigma}$$

kullanılması veri dağılımını korumak ve aykırı değerlerin etkisini azaltmak açısından matematiksel olarak daha iyi durabilirdi ama isterlerdi olmadigi icin kullanmadik.

III. Yöntem

Yöntem3.1. K-En Yakın Komşu (KNN) ve Ağırlik atama KNN algoritması, tahmin edilecek yeni verinin, eğitim setindeki verilere olan Öklid mesafesini hesaplayarak en yakın K adet komşunun "K" çoğunluk sınıfını atar.Algoritmik İyileştirme (Bizim dusundugumuz): Öklid formülü kullanıldığında, cinsiyetler arası fark (1 birim) ile harcama tutarları arasındaki fark algoritmada eşit ağırlığa sahip olur ,yani bizim cinsiyetin erkek yada kadin olmasinin arasindaki fark bitinin 1 milyon ile alisveris cikan ve 0 tl kulllanan biri ile farkin aynisi oluyor. Bu nedenle projemizde agirlik vermeyi akil ettik kullanılmıştır. Fiyatın, müşterinin ne alacağını belirlemede cinsiyete göre daha ayırt edici bir özellik olduğu analiz edilmiş ve Öklid formülünde fiyata Pe 2.5 katsayısı verdik.KNN Yalancı Kodu onumuzdeki gibidir:

3.2. Karar Ağacı , veriyi en homojen alt gruplara ayırmak için Entropi ve Bilgi Kazancı degiskenleri kullanır. Projede ağacın aşırı öğrenmesininin onune gecmek lazim bunu icin de maximum derinlik degeriyle oynamak lazim ve onu 6 olarak belirliyoruz.



IV. Nesneye Yönelik Programlama (OOP) Prensipleri

4. Nesneye Yönelik Programlam, Java'nın sunduğu NYP prensipleri kod yazarken kullanilmisir;Kapsülleme: alicilarin dataylarindan sınıfındaki değişkenlein hepsi private olarak tanımldik, verilere sadece public metotları ile kontrollü erişim sağladik.Soyutlama ve Arayüz :arayuz kismi icin IClassifier algoritmalarin standart bir sekil icinde kullandik.Kalıtım : Ana algoritmalarda ayni seyi birden fazla kez yazmamamiza yardimci oluyor bu sayede kodun hem okunabilirligi hemde uzunlugu cok uzamiyor, Karar Ağacı ve KNN bu sınıftan türetildi zaten .Çok Biçimlilik -polymorphism: mainde modellerimiz <IClassifier>listesi icinde tutuluyor ve calisma aninda hangi algoritmayi egiitmek isterse dinamik olarak ona karar verip onu seciyor .

V. Deneysel Sonuçlar

Algoritmalar K=6 ve Max Depth=6 parametreleriyle çalıştırılmış olup test verisi üzerindeki basari yuzdeleri alttaki gibidir:

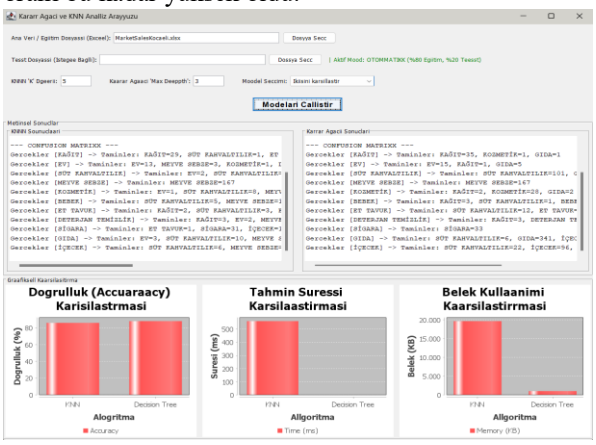
Algoritma	Doğruluk	Tahmin S.	Bellek K.
KNN	83.58%	566 ms	39.128 KB
Decision Tree	88.66%	1 ms	900 KB

VI. Değerlendirme

Bu sonuçlara baktığımızda elimize 2 tur sonuç gelir:

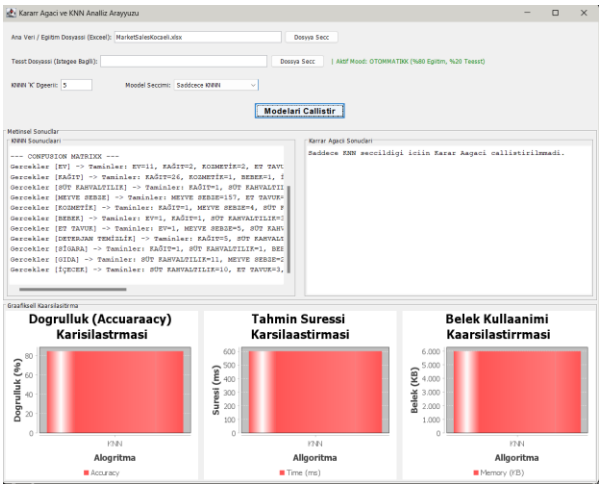
Zaman ve Bellek: Karar Ağacı tahmin yapmak istediğinde önceden belli olan if elseleri kullanarak çok hızlı 1 ms gibi çok az surede bitiriyor ve tahmin ediyor ayrıca minimum bellekle tamamlıyor ama KNN, tembel öğrenici olarak adlandırılan yöntemi kullandığı için test aşamasında her bir test verisi için binlerce eğitim verisiyle Öklid mesafesi hesaplamak zorunda kaldığı için 566 ms sürmüş ve yüksek bellek agaca nisbet cok buyuk alan kapliyor (39m).

Yüksek Doğruluğun Kaynağı : Algoritmaların %80'in üzerinde yüksek başarı göstermesinin ana sebebi, model girdileri arasında müşterinin "Marka" tercihinin de bulunmasıdır, belki bu data olmasa bu makina 50%lerde kalirdi. Müşterinin belirli bir marka alması , ürün kategorisini neredeyse kusursuz bir şekilde ele vermektedir. bizim ai müşteri id gibi gereksiz bilgilerden ziyade brend gibi doğruluk oranını yükselten verileri küllendiğimiz için bizi doğruluk oranı bu kadar yüksek oldu.



VII. Sonuç

Bu proje ile Java'da sadece veri cekme ve kosterme de hazir kutuphane kullanilmistir, sıfırdan makine öğrenmesi algoritmaları başarıyla yuksek dogruluk oraniyla geliştirdik ve NYP prensipleri cok etkili bir sekilde uyguladik. Karar ağaçlarının yapısından dolayi hızı ile KNN algoritmasının mesafe tabanlı maliyeti arasındaki farklar pratik verilerle kanıtlanmıştır.en son geliştirdigimiz GUI ile sonuçlar interaktif ve görsel olarak sunulabilir hale getirilmiştir,bizim gorsel yuzdeler ,zaman maliyeti ,bellek maliyeti karsilastiriliyor.



KNN KABA KOD

```
ONDALIK w_fiyt = 2.5
ONDALIK w_cns = 1.0
ONDALIK w_mrka = 1.0

FONKSIYON Egit gelenVeri
    egtimSeti = gelenVeri
    minFiyt = BulEnKucuk egtimSeti
    maxFiyt = BulEnBuyuk egtimSeti
FONKSIYON SONU

FONKSIYON MesafeOlc hdf_kisi egt_kisi
    fark_fiyt = hdf_kisi.fiyt - egt_kisi.fiyt / maxFiyt -
minFiyt
    fark_cns = hdf_kisi.cinsiyet - egt_kisi.cinsiyet
    fark_mrk = hdf_kisi.mrka - egt_kisi.mrka

    fiytCarpim = w_fiyt * KARE fark_fiyt
    cnsCarpim = w_cns * KARE fark_cns
    mrkCarpim = w_mrka * KARE fark_mrk

    DONDUR KAREKOK fiytCarpim + cnsCarpim +
mrkCarpim
FONKSIYON SONU

FONKSIYON TahminYap yni_kisi k_deger
    LISTE mList = BOS

    HER egt_veri ICINDE egtimSeti DONGUSU
        uzaklik = MesafeOlc yni_kisi egt_veri
        mList.EKLE uzaklik egt_veri.katgori
    DONGU SONU

    mList = Sirala mList
    DONDUR EnCokOyAlan mList ILK k_deger
FONKSIYON SONU
```

KARAR AĞACI KABA KOD

```
FONKSIYON EntropiBul veriLst
    EGER veriLst BOS ISE DONDUR 0.0
    ONDALIK ntrpi = 0.0

    HER katg ICINDE veriLst DONGUSU
        p_olaslk = katg.Sayi / veriLst.UZUNLUK
        ntrpi = ntrpi - p_olaslk * LOG_IKILIK p_olaslk
    DONGU SONU

    DONDUR ntrpi
FONKSIYON SONU

FONKSIYON BilgiKazanci bulVeri testOzlk tml_ntrpi
    SOZLUK altKumeler = VeriParcala bulVeri testOzlk
    ONDALIK yniOrtNtrpi = 0.0

    HER altLst ICINDE altKumeler.DEGERLER
DONGUSU
        dalAgirlik = altLst.UZUNLUK /
bulVeri.UZUNLUK
        yniOrtNtrpi = yniOrtNtrpi + dalAgirlik *
EntropiBul altLst
    DONGU SONU

    DONDUR tml_ntrpi - yniOrtNtrpi
FONKSIYON SONU

FONKSIYON AgacKur gnclVeri klnOzlkler
gnclDrnlik
    EGER EntropiBul gnclVeri == 0.0 VEYA gnclDrnlik
>= max_drnlik ISE
        DONDUR YeniYaprak EnCokGecenKatg
gnclVeri
    EGER SONU

    METIN iyiOzlk = EnYuksekKazancBul gnclVeri
klnOzlkler
```

```
    DUGUM yniDugum = YeniKuralDugumu iyiOzlk
    SOZLUK altParcalar = VeriParcala gnclVeri iyiOzlk
    klnOzlkler = klnOzlkler.Cikart iyiOzlk

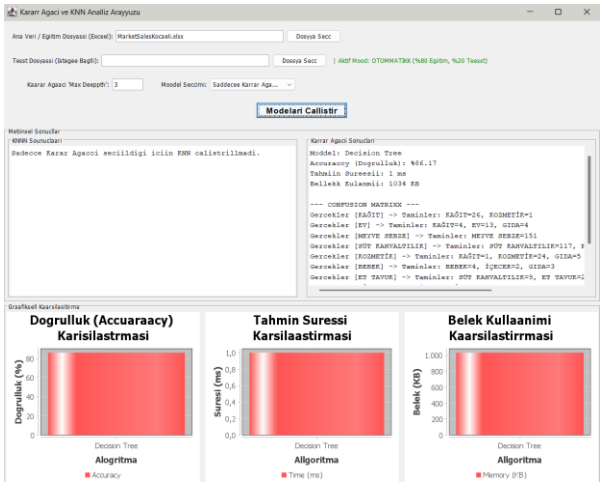
    HER prcaAdi prcaVeri ICINDE altParcalar
DONGUSU
        yniDugum.CocukEkle prcaAdi AgacKur prcaVeri
klnOzlkler gnclDrnlik + 1
    DONGU SONU

    DONDUR yniDugum
FONKSIYON SONU

FONKSIYON TaminYap yniKullanici
    DUGUM suanDgm = kokDugum

    DONGU suanDgm.YaprakMi == YANLIS
OLDUGU SURECE
        kul_deger = yniKullanici.DegerAl
suanDgm.BolunenOzelik
        suanDgm = suanDgm.Cocuklar kul_deger
    DONGU SONU

    DONDUR suanDgm.TaminEdilenKatgori
FONKSIYON SONU
```



Kaynakça

[1] T. Mitchell, Machine Learning. McGraw-Hill, 1997.

[2] J. R. Quinlan, "Induction of decision trees," Machine Learning, vol. 1, no. 1, pp. 81-106, 1986.

[3] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed. Prentice Hall, 2009.

[4] Wikipedia, "Nesne yönelimli programlama," [Çevrimiçi]. Erişim: https://tr.wikipedia.org/wiki/Nesne_y%C3%B6nelimli_programlama

[5] Wikipedia, "Object-oriented programming," [Çevrimiçi]. Erişim: https://en.wikipedia.org/wiki/Object-oriented_programming

[6] Ş. E. Şeker (Bilgisayar Kavramları), "Nesne Yönelimli Programlama (OOP) Nedir?," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=aGjb-Z72a2A

[7] Programming with Mosh, "Object-oriented Programming in 7 minutes," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=pTB0EiLXUC8

[8] freeCodeCamp.org, "Java Programming OOP Practices," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=bSrm9RXwBaI

[9] Wikipedia, "K-en yakın komşu algoritması," [Çevrimiçi]. Erişim: https://tr.wikipedia.org/wiki/K-en_yak%C4%B1n_kom%C5%9Fu_algoritmas%C4%B1

[10] Wikipedia, "K-nearest neighbors algorithm," [Çevrimiçi]. Erişim: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm

[11] Ş. E. Şeker (Bilgisayar Kavramları), "K-En Yakın Komşu Algoritması (K-Nearest Neighbor)," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=MofZ2mI2wDk

[12] StatQuest with Josh Starmer, "StatQuest: K-nearest neighbors, Clearly Explained," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=HVXime0nQeI

[13] Edureka, "K-Nearest Neighbour Algorithm Using Python/Java," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=6kZ-OPD_csU

[14] Wikipedia, "Karar ağacı öğrenmesi," [Çevrimiçi]. Erişim: https://tr.wikipedia.org/wiki/Karar_a%C4%9Fac%C4%B1_%C3%B6%C4%9Frenmesi

[15] Wikipedia, "Decision tree learning," [Çevrimiçi]. Erişim: https://en.wikipedia.org/wiki/Decision_tree_learning

[16] Ş. E. Şeker (Bilgisayar Kavramları), "Karar Ağaçları (Decision Trees) ID3 Algoritması," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=1oGzH3lG2sI

[17] StatQuest with Josh Starmer, "StatQuest: Decision Trees," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=7VeUPuVNj6I

[18] Google Developers, "Let's Write a Decision Tree Classifier from Scratch," YouTube, [Çevrimiçi]. Erişim: https://www.youtube.com/watch?v=LDRbO9a6XPU